

Blueseatra - Documentation (FR)

AI-Assisted B2B Quoting SaaS Platform

FastAPI • React • Supabase / PostgreSQL

Application multi-tenant (FastAPI + React + **Supabase/PostgreSQL**) qui transforme des demandes clients (« ordres de mission », « demandes de devis » au format PDF/image/texte) en **devis Pro Forma professionnels**, grâce à l'extraction IA, un catalogue de prix flexible et un éditeur de devis avancé avec génération PDF conforme à un gabarit métier.

Table des matières

- 1 Présentation
- 2 Fonctionnalités
- 3 Architecture & stack technique

- 4 Structure du projet
- 5 Modèle de données
- 6 Variables d'environnement
- 7 Installation locale (pas à pas)
- 8 Base de données Supabase / migration
- 9 Sécurité
- 10 Mode d'emploi (utilisateur final)
- 11 Référence API
- 12 Frontend (routes & pages)
- 13 Scripts & maintenance
- 14 Déploiement
- 15 Historique des étapes réalisées (changelog)
- 16 Dépannage / FAQ

1. Présentation

Blueseatra est une plateforme **multi-entreprises (multi-tenant)** destinée aux PME du BTP / TCE / maintenance. Chaque entreprise (« tenant ») dispose de son propre espace isolé : utilisateurs, catalogue de prix, demandes et devis.

Le flux métier principal :



2. Fonctionnalités

- **Authentification JWT** + multi-tenant (rôles : **owner**, **admin**, **operator**, **viewer**, **billing_admin**).
- **Extraction IA** de documents (PDF / image / texte) en données structurées (via Emergent LLM).
- **Catalogue « ouvert »** : import CSV de **n'importe quel format de colonnes** (auto-détection des champs + mapping manuel ajustable), stockage de toutes les colonnes en attributs dynamiques.
- **Gestion des catalogues** : versions, activation/désactivation, suppression, code client éditable.
- **Éditeur de devis professionnel** : lignes typées (Main d'œuvre, Matériel, Note, Saut de page), TVA par ligne, marge masquée sur le PDF, quantités numériques, sélecteur d'articles depuis le catalogue.
- **Génération PDF Pro Forma** conforme à un gabarit métier précis (ReportLab).
- **Profil entreprise** (raison sociale, SIRET, IBAN, mentions, conditions...) injecté dans le PDF.
- **Journal d'audit** de toutes les actions sensibles.
- **Internationalisation** FR / EN (react-i18next).

3. Architecture & stack technique

Couche	Technologie
Frontend	React 18, React Router, Tailwind CSS, Shadcn/UI (Radix), lucide [■] react, Sonner, axios, react [■] i18next
Backend	FastAPI (Python 3.11), Uvicorn, Pydantic v2
Base de données	Supabase PostgreSQL 17 (via SQLAlchemy 2 async + asyncpg)
IA / LLM	emergentintegrations (clé universelle Emergent : OpenAI / Anthropic / Google)
PDF	ReportLab
Auth	JWT (python [■] jose) + hachage bcrypt (passlib)
Chiffrement	cryptography (Fernet) pour les secrets au repos
Process mgr	supervisor (backend + frontend)

Schéma d'exécution

- **Frontend (port 3000)** → appelle le backend via **REACT_APP_BACKEND_URL** avec le préfixe **/api**.
- **Backend (port 8001)** lié à **0.0.0.0:8001**. Toutes les routes sont préfixées par **/api**.
- **Ingress Kubernetes** : route **/api/*** → backend (8001), tout le reste → frontend (3000).
- **Backend** → **PostgreSQL** : connexion directe (asyncpg) via **DATABASE_URL** (Supabase Transaction Pooler).

■ ■ **Particularité importante** : l'application **n'utilise pas l'API REST PostgREST de Supabase**. Elle se connecte directement à PostgreSQL avec le rôle propriétaire. Un **adaptateur** (**pg_adapter.py**) expose une API compatible « motor/MongoDB » au[■]dessus de SQLAlchemy, ce qui a permis de migrer la base **sans réécrire** la logique métier de **server.py**.

4. Structure du projet

```
/app
  ■■■ backend/
  ■   ■■■ server.py           # Application FastAPI : routes, auth, logique métier
  ■   ■■■ ai_service.py      # Extraction IA (Emergent LLM)
  ■   ■■■ matching.py        # Rapprochement lignes <-> catalogue + calcul des totaux
  ■   ■■■ pdf_service.py     # Génération du PDF Pro Forma (ReportLab)
  ■   ■■■ database.py        # Moteur SQLAlchemy async (lit DATABASE_URL)
  ■   ■■■ models_sql.py      # 14 modèles ORM (JSONB pour les données dynamiques)
  ■   ■■■ pg_adapter.py      # Adaptateur API motor-compatible => SQLAlchemy/PostgreSQL
  ■   ■■■ migrate_mongo_to_supabase.py # Script de migration MongoDB => Supabase (one-shot, idempotent)
  ■   ■■■ enable_rls.py      # Active RLS + policy deny-all sur toutes les tables
  ■   ■■■ requirements.txt   # Dépendances Python (gérées via pip freeze)
  ■   ■■■ SUPABASE_MIGRATION.md # Guide dédié à la migration
  ■   ■■■ .env               # Secrets & configuration (NON versionné)
  ■■■ frontend/
  ■   ■■■ src/
  ■     ■■■ App.js           # Définition des routes
  ■     ■■■ i18n.js          # Traductions FR / EN
  ■     ■■■ lib/api.js       # Client axios (base = REACT_APP_BACKEND_URL + /api)
  ■     ■■■ pages/           # Landing, Auth, Dashboard, Requests, Catalogs, QuoteEditor...
  ■     ■■■ components/ui/   # Composants Shadcn/UI
  ■     ■■■ package.json     # Dépendances JS (gérées via yarn)
  ■     ■■■ .env             # REACT_APP_BACKEND_URL (NE PAS modifier)
  ■■■ design_guidelines.md
  ■■■ README.md              # <- ce document
```

5. Modèle de données

14 tables PostgreSQL (clés primaires en **UUID** string). Les champs dynamiques/imbriqués sont stockés en **JSONB**.

Table	Rôle	Champs JSONB
<code>users</code>	Comptes utilisateurs (email unique, <code>password_hash</code> bcrypt)	—
<code>tenants</code>	Entreprises / espaces	—
<code>tenant_users</code>	Appartenance utilisateur <-> tenant + rôle	—
<code>catalogs</code>	Catalogues (nom, <code>client_code</code> , <code>active_version_id</code>)	—
<code>catalog_versions</code>	Versions d'un catalogue (statut, mapping...)	<code>columns</code> , <code>mapping</code>
<code>pricing_items</code>	Articles de catalogue (label, prix, TVA, marge...)	<code>suppliers</code> , <code>attributes</code>
<code>requests</code>	Demandes client + extraction IA	<code>extracted</code>
<code>quotes</code>	Devis (client, site, totaux...)	<code>lines</code> , <code>meta</code> , <code>pricing_snapshot</code>
<code>quote_versions</code>	Historique des versions de devis	<code>snapshot</code>
<code>import_jobs</code>	Journaux d'import CSV	—
<code>import_errors</code>	Erreurs ligne par ligne d'import	<code>raw</code>
<code>audit_logs</code>	Journal d'audit	<code>meta</code>
<code>settings_integrations</code>	Réglages IA/n8n par tenant (clé IA chiffrée)	—
<code>company_profiles</code>	Profil entreprise pour le PDF	—

Isolation multi-tenant : chaque requête filtre par `tenant_id`, et toute modification est précédée d'un contrôle de propriété (un identifiant d'un autre tenant renvoie 404).

6. Variables d'environnement

backend/.env

Variable	Description
<code>DATABASE_URL</code>	URI Transaction Pooler Supabase (port 6543). Obligatoire .
<code>SUPABASE_URL</code>	URL du projet Supabase (<a href="https://<ref>.supabase.co">https://<ref>.supabase.co)
<code>SUPABASE_ANON_KEY</code>	Clé publique <code>anon</code> (réservée à un usage futur Auth/Storage)
<code>SUPABASE_SERVICE_ROLE_KEY</code>	Clé <code>service_role</code> (ultra-sensible , serveur uniquement)
<code>JWT_SECRET</code>	Secret de signature JWT — doit être fort (>= 32 car., non par défaut)
<code>APP_ENCRYPTION_KEY</code>	Clé Fernet (base64) pour chiffrer les secrets au repos
<code>EMERGENT_LLM_KEY</code>	Clé universelle Emergent (OpenAI/Anthropic/Google)
<code>MAX_UPLOAD_SIZE</code>	(optionnel) Taille max d'upload en octets (défaut 15 Mo)
<code>CORS_ORIGINS</code>	(optionnel) Origines autorisées, séparées par virgule (ex. https://blueseatra.com)
<code>MONGO_URL, DB_NAME</code>	Conservés pour le script de migration (source MongoDB)

■ **Ne jamais committer** `.env`. Le backend **refuse de démarrer** si `JWT_SECRET` est faible/par défaut.

frontend/.env

Variable	Description
REACT_APP_BACKEND_URL	URL publique du backend. NE PAS modifier (géré par la plateforme).

Générer des secrets forts

```
# JWT_SECRET
python -c "import secrets; print(secrets.token_urlsafe(48))"
# APP_ENCRYPTION_KEY (Fernet)
python -c "from cryptography.fernet import Fernet; print(Fernet.generate_key().decode())"
```

7. Installation locale (pas à pas)

Prérequis

- Python 3.11
- Node.js 20 + Yarn (ne pas utiliser npm)
- Un projet Supabase (gratuit) OU un PostgreSQL accessible

1) Récupérer le code

```
git clone <votre-repo> blueseatra && cd blueseatra
```

2) Backend

```
cd backend
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
pip install emergentintegrations --extra-index-url https://d33sy5i8bnduwe.cloudfront.net/simple/
```

Créez backend/.env (voir section 6) en renseignant au minimum

DATABASE_URL, JWT_SECRET, APP_ENCRYPTION_KEY, EMERGENT_LLM_KEY.

3) Initialiser la base

```
# Crée le schéma + active la RLS
python migrate_mongo_to_supabase.py --create-tables
# (Optionnel) migrer des données existantes depuis MongoDB
python migrate_mongo_to_supabase.py
```

4) Lancer le backend

```
uvicorn server:app --host 0.0.0.0 --port 8001 --reload
```

5) Frontend

```
cd ../frontend
yarn install
# frontend/.env : REACT_APP_BACKEND_URL=http://localhost:8001
yarn start
```

Application disponible sur <http://localhost:3000>.

En production sur cette plateforme, **supervisor** gère les deux services : `sudo supervisorctl restart backend frontend`. Le hot-reload est actif (pas besoin de redémarrer pour un simple changement de code, seulement pour

.env/dépendances).

8. Base de données Supabase / migration

Connexion (point critique)

Utilisez **impérativement** l'URI **Transaction Pooler** (port **6543**), récupérée dans

Supabase -> **Connect** -> onglet ***Transaction Pooler*** :

```
postgresql://postgres.<ref>:<MOT_DE_PASSE>@aws-0-<region>.pooler.supabase.com:6543/postgres
```

■ L'URI « Direct Connection » (db.<ref>.supabase.co:5432) **ne fonctionne pas** dans cet environnement (résolution IPv4 indisponible). Le moteur asyncpg est configuré avec `statement_cache_size=0` (obligatoire avec le pooler en mode `*transaction*`).

Étapes de migration MongoDB -> Supabase

```
cd backend
python migrate_mongo_to_supabase.py --create-tables # 1) schéma + RLS
python migrate_mongo_to_supabase.py # 2) copie des données (idempotent)
```

Le script :

- ne supprime **rien** dans MongoDB (sécurité) ;
- est **idempotent** (upsert par clé primaire — relançable sans risque) ;
- projette automatiquement chaque document Mongo sur les colonnes du modèle.

Détails complémentaires dans `backend/SUPABASE_MIGRATION.md`.

9. Sécurité

Mesures en place (auditées) :

Domaine	Mesure
JWT	Secret aléatoire fort ; refus de démarrage si secret faible/par défaut/<32 car.
Mots de passe	Hachage bcrypt (passlib) ; jamais stockés en clair.
Rôles	Le rôle est re✓ifié en base (<code>tenant_users</code>), jamais lu depuis le token.
Isolation multi■tenant	Filtrage systématique par <code>tenant_id</code> + contrôle de propriété (404 sinon).
Secrets au repos	Clé IA tenant chiffrée (Fernet) , préfixe <code>enc::</code> ; jamais renvoyée au client.
CORS	<code>allow_credentials</code> activé uniquement avec des origines explicites (jamais en wildcard).
Upload	Limite 15 Mo (HTTP 413) sur documents et CSV.
PostgreSQL / Supabase	RLS activée + <code>policy deny_all</code> + privilèges <code>anon/authenticated</code> révoqués sur les 14 tables. L'API REST publique ne peut donc rien lire/écrire.

L'app passe par une **connexion PostgreSQL directe** (rôle propriétaire qui **contourne** la RLS), donc la RLS n'a aucun impact sur le fonctionnement, mais ferme la porte à l'API publique.

Recommandations restantes (non bloquantes)

- Rate limiting sur `/api/auth/login` (anti brute force).
- Token PDF court à usage unique (le téléchargement utilise actuellement `?token=`).
- Politique de mot de passe renforcée (8+ caractères, complexité).
- En prod : définir `CORS_ORIGINS=https://blueseatra.com`.

10. Mode d'emploi (utilisateur final)

1 **Créer un compte** (`/signup`) -> un espace entreprise (tenant) est créé automatiquement, avec un catalogue de démonstration.

1 **Importer son catalogue** (Catalogues -> *Importer CSV*) :

2 Étape 1 : déposer le fichier CSV (n'importe quel format de colonnes, séparateur `,` ou `;`).

3 Étape 2 : vérifier l'aperçu et le mapping auto-déTECTÉ (ajustable). Seul le champ

Libellé/Désignation est obligatoire.

1 Étape 3 : valider -> le catalogue devient actif. Toutes les colonnes d'origine sont conservées.

2 **Créer une demande** (Demandes) : téléverser un PDF/image d'« ordre de mission » ou coller du texte -> l'IA en extrait les données structurées.

1 **Générer un devis** : depuis une demande, créer un **brouillon** ; les lignes sont pré-rapprochées avec le catalogue.

1 **Éditer le devis** (Éditeur) : ajouter/supprimer des lignes (Main d'œuvre, Matériel, Note, Saut de page), régler quantités/TVA/marge, choisir des articles du catalogue.

1 **Renseigner le profil entreprise** (Paramètres) : raison sociale, SIRET, IBAN, mentions...

2 **Valider** le devis puis **télécharger le PDF Pro Forma**.

3 **Gérer l'équipe** (Membres) : inviter des utilisateurs, définir les rôles.

4 **Suivre l'activité** (Journal d'audit).

11. Référence API

Toutes les routes sont préfixées par `/api` et (sauf signup/login) protégées par

`Authorization: Bearer <token>`.

Authentication & membres

Méthode	Route	Description
POST	<code>/api/auth/signup</code>	Création compte + tenant
POST	<code>/api/auth/login</code>	Connexion (retourne un token)
GET	<code>/api/auth/me</code>	Profil + tenants accessibles
POST	<code>/api/auth/switch-tenant/{tenant_id}</code>	Changer de tenant actif
GET / POST	<code>/api/members</code>	Lister / inviter des membres
PATCH	<code>/api/members/{user_id}</code>	Modifier un rôle

Paramètres & profil

Méthode	Route	Description
GET / PUT	/api/settings/integrations	Réglages IA/n8n (clé IA chiffrée, jamais renvoyée)
GET / PUT	/api/company-profile	Profil entreprise (PDF)

Demandes

Méthode	Route	Description
POST	/api/requests	Créer une demande (fichier/texte)
GET	/api/requests	Lister
GET	/api/requests/{id}	Détail
POST	/api/requests/{id}/process	(Re)lancer l'extraction IA

Catalogues

Méthode	Route	Description
GET	/api/catalogs	Lister (avec versions)
GET	/api/catalogs/{id}/items	Articles + colonnes dynamiques + mapping
GET	/api/catalog-template.csv	Modèle CSV à télécharger
POST	/api/catalogs/import/preview	Aperçu + mapping auto■déecté
POST	/api/catalogs/import	Import (mapping ajustable)
GET	/api/import-jobs/{job_id}/errors	Erreurs d'un import
POST	/api/catalogs/{id}/activate/{version_id}	Activer une version
PATCH	/api/catalogs/{id}	Modifier (ex. client_code)
POST	/api/catalogs/{id}/deactivate	Désactiver
DELETE	/api/catalogs/{id}	Supprimer (cascade)
GET	/api/catalog/active	Catalogue actif (pour le sélecteur de devis)

Devis

Méthode	Route	Description
POST	/api/quotes/draft	Créer un brouillon depuis une demande
GET	/api/quotes	Lister
GET	/api/quotes/{id}	Détail
PATCH	/api/quotes/{id}	Éditer (lignes, TVA, marge...)
POST	/api/quotes/{id}/validate	Valider (fige une version)
POST	/api/quotes/{id}/send	Marquer envoyé
GET	/api/quotes/{id}/pdf	Télécharger le PDF Pro Forma

Divers

Méthode	Route	Description
GET	/api/dashboard	Indicateurs
GET	/api/audit	Journal d'audit
GET	/api/	Healthcheck

Exemple cURL

```
# Connexion
curl -X POST "$BACKEND/api/auth/login" -H "Content-Type: application/json" \
-d '{"email":"demo@blueseatra.com","password":"secret123"}'
# Lister les catalogues
curl "$BACKEND/api/catalogs" -H "Authorization: Bearer <TOKEN>"
```

12. Frontend (routes & pages)

Route	Page	Accès
/	Landing	Public
/login, /signup	Authentification	Public
/app	Tableau de bord	Protégé
/app/requests, /app/requests/:id	Demandes / détail	Protégé
/app/catalogs, /app/catalogs/import	Catalogues / import	Protégé
/app/quotes, /app/quotes/:id	Devis / éditeur	Protégé
/app/members	Membres	Protégé
/app/audit	Journal d'audit	Protégé
/app/settings	Paramètres / profil	Protégé
/app/billing	Facturation	Protégé

- Internationalisation : `frontend/src/i18n.js` (FR/EN).
- Client API : `frontend/src/lib/api.js` (base `REACT_APP_BACKEND_URL` + `/api`, injection du token).
- Vérification de build : `npx esbuild src/ --loader:.js=jsx --bundle --outfile=/dev/null` (jamais `npm`).

13. Scripts & maintenance

Script	Usage
<code>python migrate_mongo_to_supabase.py</code> <code>--create-tables</code>	Crée le schéma + active la RLS
<code>python migrate_mongo_to_supabase.py</code>	Migre les données MongoDB -> Supabase (idempotent)
<code>python enable_qls.py</code>	(Re)applique RLS + policy <code>deny_all</code> + révocation <code>anon/authenticated</code>
<code>pip freeze > requirements.txt</code>	Met à jour les dépendances backend (après <code>pip install</code>)
<code>yarn add <pkg></code>	Ajoute une dépendance frontend
<code>sudo supervisorctl status / restart backend</code> <code>frontend</code>	Gestion des services
<code>tail -n 100 /var/log/supervisor/backend.*.log</code>	Logs backend

14. Déploiement

- 1 **Vérifier** la « déployabilité » (aucun secret en dur, ports, CORS, build).
- 2 **Déployer** via le bouton **Deploy** d'Emergent (Preview -> Deploy -> Deploy Now).
- 3 **Domaine personnalisé** (`blueseatra.com`) : Dashboard -> Connect -> intégration **Entri**
(configuration DNS guidée).

1 En production : définir `CORS_ORIGINS=https://blueseatra.com` et conserver

`JWT_SECRET` / `APP_ENCRYPTION_KEY` hors dépôt Git.

Contraintes plateforme : backend lié à `0.0.0.0:8001`, routes `/api/*`, services via supervisor, ne **jamais** modifier `REACT_APP_BACKEND_URL` ni la configuration d'ingress.

15. Historique des étapes réalisées (changelog)

Phase 1■2 — MVP & socle

- Auth JWT, multi■tenant, routage frontend, tableau de bord.
- Extraction IA des demandes (PDF/image/texte) -> données structurées.
- Génération PDF Pro Forma conforme au gabarit métier.

Phase 3 — Éditeur de devis professionnel

- Lignes typées (Main d'œuvre, Matériel, Note, Saut de page), TVA par ligne, marge masquée au PDF, quantités numériques, adaptation A4, sélecteur d'articles depuis le catalogue.
- Import catalogue hiérarchique (Famille -> Article -> Fournisseur).

Phase 4 — Catalogue « ouvert » (CSV dynamique)

- Auto■détection des colonnes (synonymes FR/EN) + mapping ajustable dans l'assistant d'import.
- Lecture robuste (séparateur `,`/`;`/`tab`, encodages, décimales `12,50`).
- Conservation de **toutes** les colonnes en `attributes` (JSONB) ; affichage en colonnes dynamiques + détails.
- Boutons **Désactiver** / **Supprimer** un catalogue, **code client éditable**, alignement UI.
- Correctif d'affichage des champs numériques (Qté/Prix/Marge) dans l'éditeur.

Audit de sécurité

- `JWT_SECRET` fort + refus du secret par défaut ; CORS durci ; limite d'upload 15 Mo ;

chiffrement Fernet de la clé IA ; vérification de l'isolation multi■tenant.

Migration vers Supabase (PostgreSQL)

- Connexion via Transaction Pooler (asyncpg, `statement_cache_size=0`).
- 14 modèles SQLAlchemy (JSONB pour les données dynamiques) + schéma créé.
- **Adaptateur** `pg_adapter.py` (API motor■compatible) -> `server.py` quasi inchangé.
- Migration des données existantes (idempotente, sans suppression de MongoDB).
- Tests : backend 60/61, frontend 95 %, global 97 % (1 bug `update_one` corrigé).

Durcissement Supabase

- RLS activée sur les 14 tables, policy `deny_all`, privilèges `anon/authenticated` révoqués

-> Security Advisor « 0 erreur / 0 warning / 0 info ».

16. Dépannage / FAQ

Le backend refuse de démarrer (`JWT_SECRET ...`)

-> Définissez un `JWT_SECRET` fort (≥ 32 caractères) dans `backend/.env`.

`DATABASE_URL` not set / erreurs de connexion DB

-> Renseignez l'URI **Transaction Pooler** (port 6543) avec le **mot de passe** réel. La Direct Connection (5432) ne fonctionne pas ici.

Erreur `prepared statement` avec le pooler

-> Déjà géré (`statement_cache_size=0`). Si vous changez de moteur, conservez ce réglage en mode `*transaction*`.

HTTP 413 à l'upload

-> Fichier > 15 Mo. Ajustez `MAX_UPLOAD_SIZE` si nécessaire.

Supabase Security Advisor signale des soucis RLS

-> Lancez `python enable_rls.py`, puis « Rerun linter » dans le dashboard.

Les anciens tokens ne fonctionnent plus

-> Normal après une rotation de `JWT_SECRET` : reconnectez-vous.

Ne pas utiliser `npm` — uniquement `yarn`. Ne **jamais** lancer les serveurs à la main :
passer par **supervisor**.

`_Blueseatra` — documentation maintenue dans `/app/README.md`. Pour la migration de base de données, voir aussi `backend/SUPABASE_MIGRATION.md`.